Internship Project Detailed Documentatio

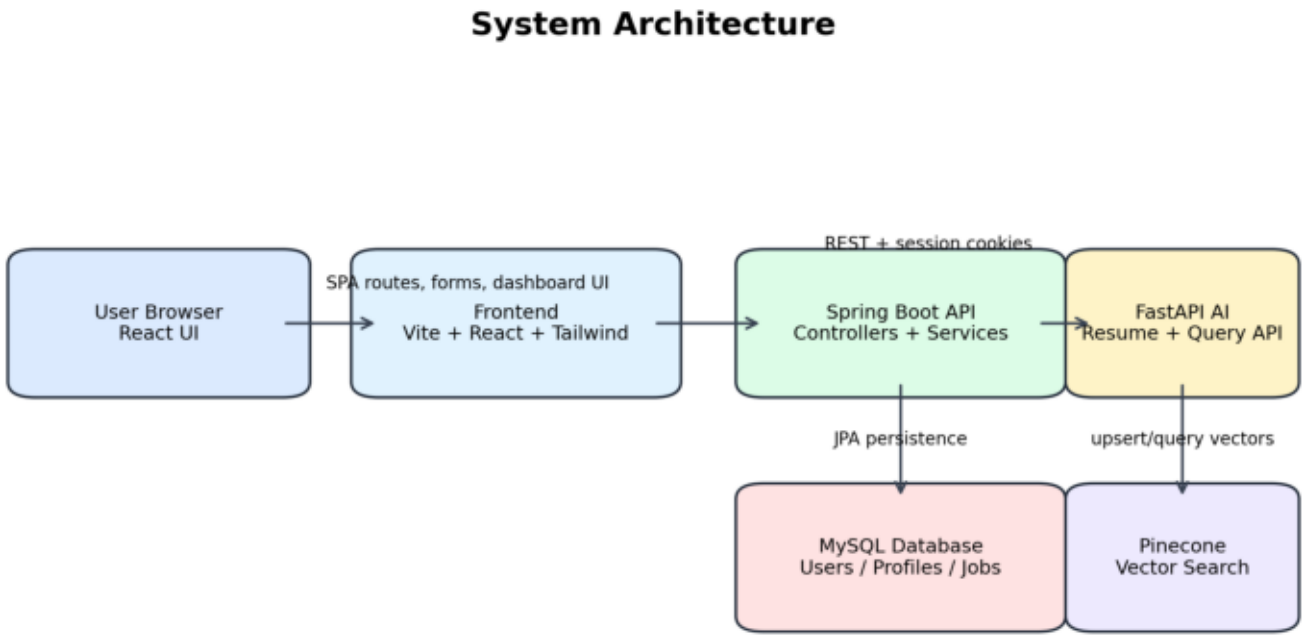
Repository-wide technical documentation generated from code inspection

Coverage

- Spring Boot backend architecture and API routes
- React frontend routes, components, and state handling
- FastAPI resume analysis and semantic matching service
- Workflow diagrams, implementation notes, and interview talking points

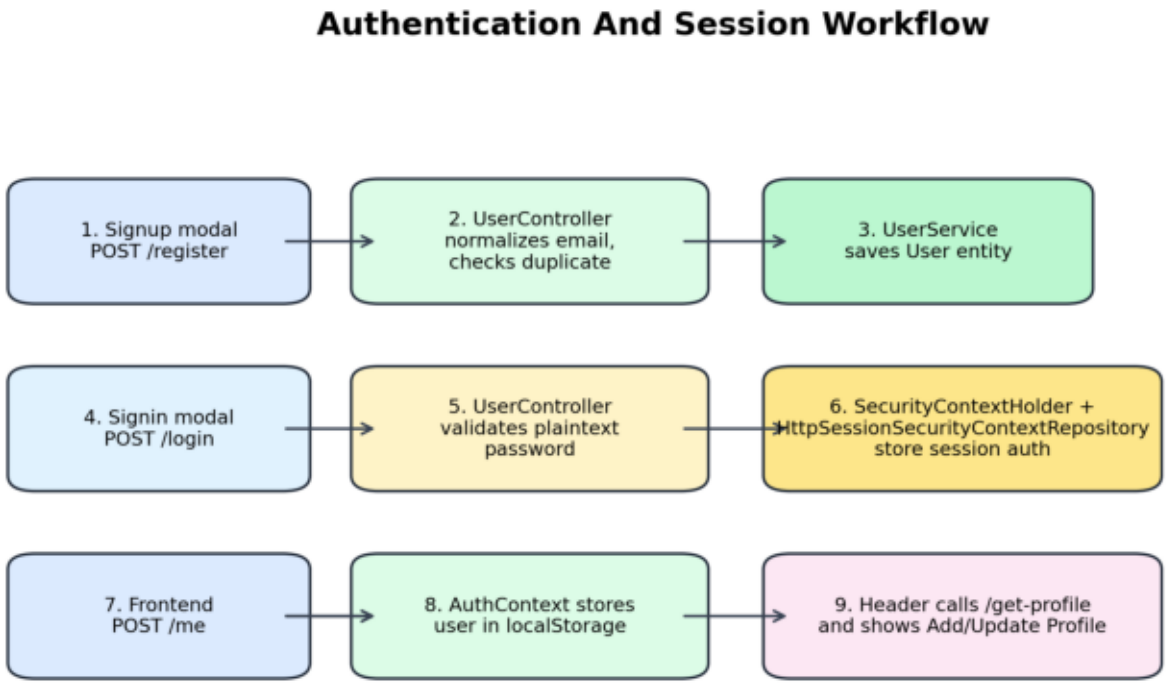
Generated on 2026-04-18 from the local repository at e:\Internship

Architecture Diagram



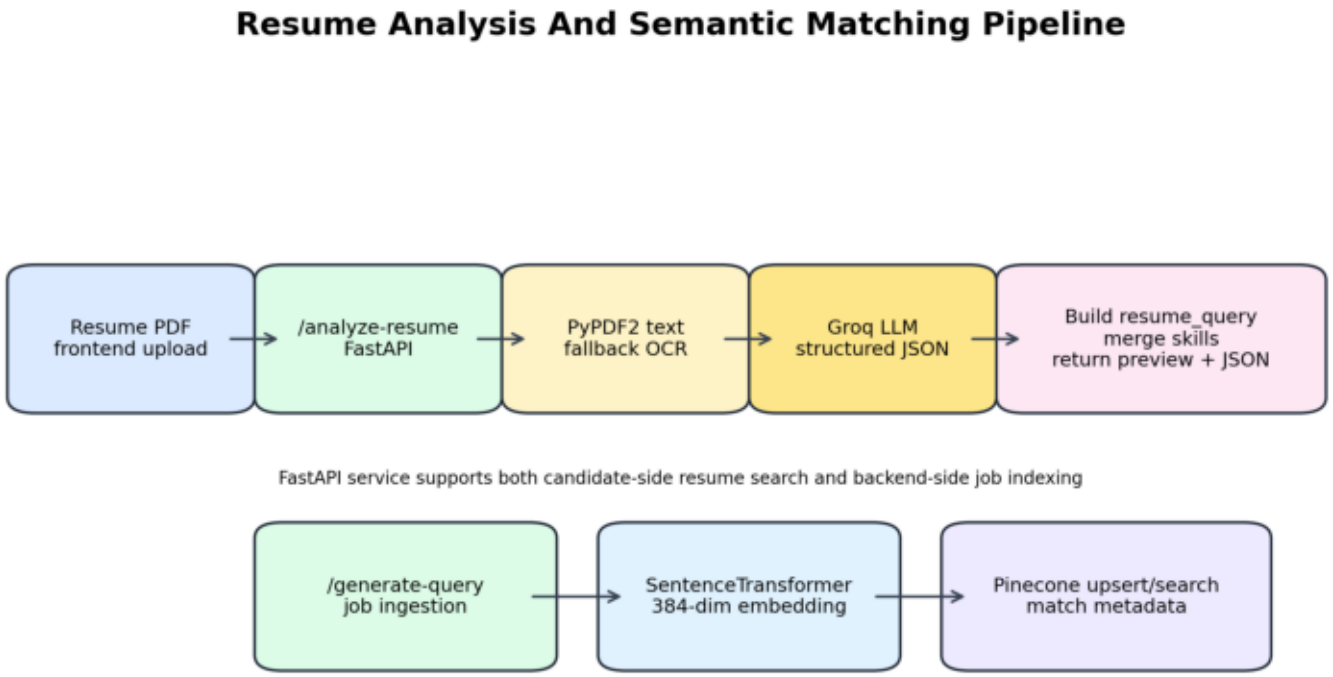
High-level view of how the browser, React app, Spring Boot API, MySQL database, FastAPI service, and Pinecone vector store interact.

Authentication Workflow



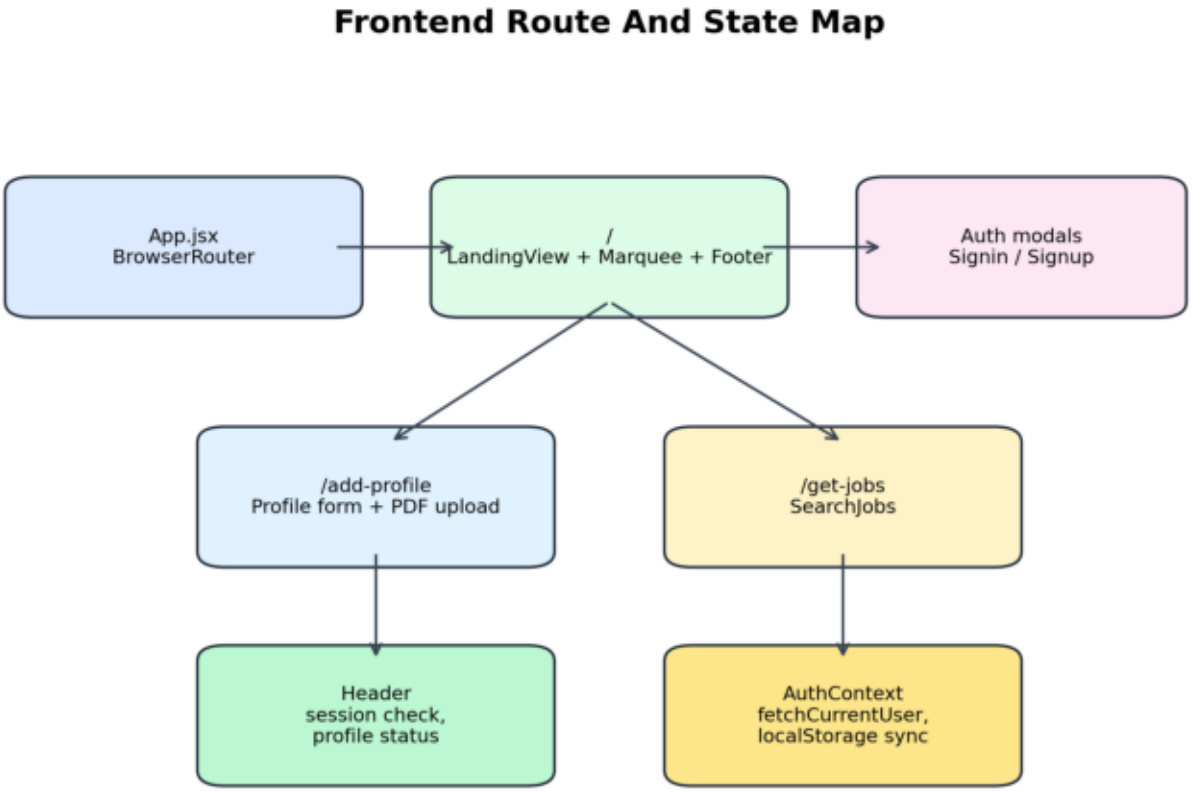
Session creation begins in the signin modal and ends with frontend state hydration through `/me` plus profile-state checks through `/get-profile`.

Resume And Matching Pipeline



The AI service handles both resume understanding for candidates and vector indexing for jobs.

Frontend Route Map



This route map shows how the SPA composes the landing page, profile form, search page, header, and auth context.

Project Overview

This repository contains a multi-service internship and job matching platform. The codebase is split into three product-facing parts:

- A Spring Boot backend in `backend/demo` that exposes REST endpoints for authentication, profile creation, profile lookup, and job creation.
- A React frontend in `frontend` that provides the landing page, sign-in and sign-up modals, profile submission screen, and resume-based job search page.
- A FastAPI-based AI service in `resume` that extracts resume text, enriches it with an LLM, generates semantic search queries, creates sentence-transformer embeddings, and stores or searches vectors in Pinecone.

There are also supporting files:

- `Jenkinsfile` defines the CI/CD pipeline.
- `README.md` contains a high-level project introduction.
- `h.go` and `resume/temp.py` look like practice or sandbox files rather than production features.

Interview-ready one-line summary:
This is a full-stack internship recommendation system where the Java backend manages user and job records, the Python service handles semantic understanding and vector search, and the React frontend ties both services together into a candidate-facing experience.

Repository Structure

The repository contains 116 non-generated files once standard build folders and virtual environments are excluded. The main source-bearing areas are:
- `backend/demo/src/main/java/com/app/demo`: Spring Boot application code.
- `backend/demo/src/main/resources/application.properties`: runtime configuration for server port, MySQL, Hibernate, and actuator.
- `frontend/src`: React components, auth context, utility helpers, and route composition.
- `frontend/public`: 22 SVG image assets used by the marquee.
- `resume/main.py`: the AI service entry point with all route and helper logic in one file.
- `resume/extracted_texts`: generated outputs from resume parsing runs.

The backend follows a classic layered structure:
- `Controller`: HTTP endpoints.
- `Service`: business logic plus outgoing HTTP calls.
- `Repository`: Spring Data JPA persistence interfaces.
- `Entity`: database models.
- `DTO`: transport objects between controller, service, and external APIs.
- `Config`: beans and framework configuration.

The frontend is component-driven:
- `App.jsx` defines the route tree.
- `context/AuthContext.jsx` is the main shared state for session bootstrap and current user identity.
- `components/home` holds landing and discovery UI.
- `components/layout` holds the header, footer, and auth modals.
- `components/dashboard` currently contains placeholder components returning `null`.
- `components/ui` contains Radix-style reusable primitives built with class-variance-authority.

Backend Runtime And Configuration

The backend is a Spring Boot 4.0.3 application started through `DemoApplication.java`.
`application.properties` sets:
- `server.port=8081`
- datasource URL `jdbc:mysql://localhost:3306/Internship`
- username `root`
- password `root`
- `spring.jpa.hibernate.ddl-auto=update`
- `spring.jpa.show-sql=true`
- `management.endpoints.web.exposure.include=*`

What this means operationally:
- The backend expects a local MySQL instance and updates the schema automatically on startup.
- SQL statements are printed to the console, which helps debugging during development.
- Actuator and Prometheus endpoints are available.
- The backend is configured for development convenience rather than production hardening.

Configuration classes:
- `AppConfig` registers a `RestTemplate` bean used by services that call other APIs.
- `SecurityConfig` disables CSRF, configures request authorization, and allows CORS from `http://localhost:5173`.
- `WebConfig` maps the physical `uploads` directory to `/uploads/**`, making uploaded files accessible if present.

Important detail:
Although security rules mention authentication, several endpoints such as `/add-profile`, `/upload`, and `/me` are also marked as `permitAll()` before a later `/me authenticated()` matcher. In Spring Security, order matters, so the earlier matcher wins. The controllers still do manual auth checks, but the route-level rule is looser than it looks.

Backend Domain Model

The backend persists three main entities.

`User`:
- Table name: `users`
- Fields: `id`, `firstName`, `lastName`, `email`, `password`
- Constraints: `email` is unique and non-null
- Relationship: one-to-one with `Profile` through `profile`

`Profile`:
- Fields: `id`, `phoneNumber`, `gender`, `state`, `user`
- Relationship owner: `@OneToOne` with `@JoinColumn(name = "user_id", nullable = false)`
- Each user can have exactly one profile in the current design.

`Job`:
- Fields: `id`, `title`, `company`, `location`, `skills`
- `skills` is declared as `List<String>`.

Interview note:
`Job.skills` is not annotated with `@ElementCollection` or a converter. In standard JPA this is incomplete and may fail schema generation or persistence depending on runtime behavior. It is an important design gap to understand if asked about persistence modeling.

Backend Repositories And DTOs

Repositories:
- `UserRepository extends JpaRepository<User, Integer>` with `findByEmail(String email)`.
- `ProfileRepository extends JpaRepository<Profile, Integer>` with `getProfileByUserId(Integer id)`.
- `JobRepository extends JpaRepository<Job, Integer>` with default CRUD behavior only.

DTOs used by the backend:
- `UserRegisterDTO`: incoming registration payload with id, firstName, lastName, email, password.
- `UserResponseDTO`: outgoing user projection with id, firstName, lastName, email.
- `LoginRequestDTO`: incoming email and password.
- `LoginResponseDTO`: outgoing login result containing message and success flag.
- `ProfileRequestDTO`: incoming phoneNumber, gender, state.
- `JobRequestDTO`: payload sent from Spring Boot to FastAPI for semantic job indexing.
- `JobResponseDTO`: backend response returned after saving a job and calling FastAPI.
- `PythonResponseDTO`: deserializes FastAPI output in the Java service layer.

Design interpretation:
The Java backend uses DTOs mainly for transport shaping, especially to avoid returning the full `User` entity directly and to translate the FastAPI contract into a Java object.

Backend Controllers And API Routes

The backend exposes these routes.

`POST /register`
- Controller: `UserController.register`
- Input: `UserRegisterDTO`
- Flow: trims and lowercases email, checks if the email already exists, calls `UserService.register`, returns `UserResponseDTO`
- Failure mode: duplicate email returns HTTP 400 with an empty body

`POST /login`
- Controller: `UserController.login`
- Input: `LoginRequestDTO`
- Flow: normalizes email, loads user by email, compares plaintext password, creates `UsernamePasswordAuthenticationToken`, saves security context into HTTP session, returns `LoginResponseDTO`
- Failure mode: missing user returns 404, wrong password returns 400

`POST /logout`
- Controller: `UserController.logout`
- Flow: clears `SecurityContextHolder` and invalidates the current session if it exists
- Output: simple success string

`POST /me`
- Controller: `UserController.getCurrentUser`
- Purpose: session bootstrap endpoint used by the frontend to recover the current user from the server-side session
- Output: `UserResponseDTO`
- Failure mode: 401 if authentication is missing or user cannot be resolved

`GET /get-profile`
- Controller: `UserController.getProfile`
- Flow: reads current email from the security context, finds the user, then calls `profileRepository.getProfileByUserId(user.getId())`
- Output: full `Profile` entity

Backend Controllers And API Routes

- Failure mode: 404 if user or profile is missing

`POST /upload`
- Controller: `ProfileController.uploadPdf`
- Input: multipart file under key `file`
- Flow: delegates to `UserService.sendPdfToFastApi`, which forwards the file to `http://localhost:8000/analyze-resume`
- Output: proxied JSON from FastAPI
- Failure mode: HTTP 500 with message and exception text

`POST /add-profile`
- Controller: `ProfileController.addProfile`
- Input: `ProfileRequestDTO`
- Flow: requires an authenticated security context, prevents duplicate profile creation, creates `Profile`, links it to the user, and saves it
- Output: plain text success message
- Failure mode: 401 if not logged in, 404 if user missing, 409 if profile already exists

`POST /create-job`
- Controller: `JobController.postJob`
- Input: `Job` entity JSON
- Flow: saves the job in MySQL, converts it to `JobRequestDTO`, calls the Python service, then returns a `JobResponseDTO`
- Output: combined persistence and semantic indexing result

Backend Services And Internal Logic

`UserService.register`
- Creates a new `User` entity manually.
- Copies fields from `UserRegisterDTO`.
- Lowercases and trims the email.
- Saves through `UserRepository`.
- Returns `UserResponseDTO`.

`UserService.sendPdfToFastApi`
- Accepts a Spring `MultipartFile`.
- Rebuilds it as a `ByteArrayResource`.
- Creates a multipart/form-data request.
- Calls `http://localhost:8000/analyze-resume` using a locally instantiated `RestTemplate`.
- Returns the raw response body as `Map<String, Object>`.

`JobService.saveJobAndGenerateQuery`
- Saves the `Job` first using JPA.
- Converts the saved entity into `JobRequestDTO`.
- Sends JSON to `http://127.0.0.1:8000/generate-query`.
- Deserializes the FastAPI result into `PythonResponseDTO`.
- Repackages the final data as `JobResponseDTO`.

Interesting design choice:
The Java backend saves the job record before confirming that vector indexing succeeded. If the Python service fails, the database may still contain the job while Pinecone does not. This creates eventual inconsistency and is a good talking point in interviews.

Unused or placeholder backend code:
- `FileStorageService.java` contains only `// Removed.`
- There is no custom password encoder or user-details service.
- There is only one minimal test, `contextLoads`.

Frontend Application Architecture

The frontend is a Vite React SPA.

Boot sequence:
- `main.jsx` mounts the React tree into `#root`.
- `AuthProvider` wraps the entire app.
- `App.jsx` uses `BrowserRouter` and conditionally shows a loading spinner while auth initialization is in progress.

Defined routes:
- `/`: landing page with `Header`, `LandingView`, `Marquee`, and `Footer`. Sign-in and sign-up render as modal overlays on top of this route.
- `/add-profile`: page containing `Header`, `Profile`, and `Footer`.
- `/get-jobs`: page containing the resume upload and semantic job results UI.

`AuthContext.jsx` is the state hub:
- Initializes `user` from `localStorage`.
- Calls backend `POST /me` with `credentials: 'include'` to restore the session on page load.
- Stores the normalized user object in local storage.
- Exposes `login`, `register`, `logout`, and `refetchUser`.

Important inconsistency:
- `fetchCurrentUser` uses backend port `8081`.
- `login` and `register` inside `AuthContext` use port `8080`.
- The auth modals themselves call `8081`.

This means there are two overlapping auth implementations in the frontend, and only one of them points at the configured backend port.

Frontend Components And User Flows

`Header.jsx`
- Shows branding, navigation, voice and language icon buttons, auth buttons, and profile-state actions.
- If a user exists, it calls `GET /get-profile` to decide whether to show `Add Profile` or `Update Profile`.
- Includes a debugging-oriented `Check /me` button that calls the backend and pops browser alerts.
- Logout calls `POST /logout`, clears frontend auth state, navigates home, and reloads the page.

`Signin.jsx`
- Modal login form.
- Calls `POST /login` on port `8081` with cookies enabled.
- Reads the response as text rather than JSON.
- On success it shows an alert, clears form fields, closes the modal, and reloads the page.
- It also writes `data.token` to local storage even though `data` is text, so the token line is ineffective.

`Signup.jsx`
- Modal registration form.
- Calls `POST /register` on port `8081`.
- On success it clears the fields but does not auto-login the user.

`Profile.jsx`
- Collects phone number, gender, state, and an `id` field, although the backend ignores `id` in the DTO.
- `handleSubmit` sends JSON to `POST /add-profile`.
- `handleUpload` sends a multipart PDF to `POST /upload`.
- The screen combines profile creation and resume analysis into one page.

`SearchJobs.jsx`
- Uploads a resume directly to the Python service instead of going through Spring Boot.
- First calls `POST /analyze-resume`.
- Builds a query from `resume_query` or a summary-plus-skills fallback.
- Then calls `GET /search`.
- Filters matches to scores greater than `0.35`.

Frontend Components And User Flows

- Renders title, company, location, skills, and the similarity score.

Landing and marketing components:
- `LandingView.jsx`: hero section, feature cards, about section, gallery, and contact cards.
- `GalleryCarousel.jsx`: auto-rotating image carousel based on remote Unsplash images.
- `Marquee.jsx` and `MarqueeItem.jsx`: animated horizontal strip of local SVG assets.
- `Footer.jsx`: quick links, candidate links, subscription input, and basic site footer.

Frontend Styling System

Tailwind is configured in `tailwind.config.js` with custom colors:
- `primary`: blue
- `secondary`: green
- `match.high`, `match.med`, `match.low`
- `text.main`, `text.muted`

Typography:
- Headings use `Outfit`.
- Body text uses `Inter`.

`src/styles/index.css` defines the visual language:
- A fixed green-blue gradient page background.
- `glass-panel` and `glass-nav` utilities for glassmorphism surfaces.
- `btn`, `btn-primary`, `btn-secondary`, `btn-success`, and auth button helpers.
- `footer-gradient`.

Reusable UI primitives:
- `Button.jsx` wraps Radix `Slot` plus CVA variants.
- `Card.jsx`, `Input.jsx`, and `Label.jsx` provide styled form building blocks.
- `utils.js` exposes `cn()` to combine class names via `clsx` and `tailwind-merge`.

Observation:
The landing area follows a richer design language than the profile and search pages, which are visually much more utilitarian. This suggests the project evolved incrementally rather than through a single design system pass.

Python AI Service Architecture

The Python service in `resume/main.py` is a large single-file FastAPI application. It combines API routing, model loading, helper functions, external service clients, and persistence-side effects.

Startup behavior:
- Loads environment variables from `.env`.
- Requires both `PINECONE_API_KEY` and `GROQ_API_KEY`.
- Uses `PINECONE_INDEX_NAME` default `quickstart`.
- Uses `PINECONE_NAMESPACE` default `job-queries`.
- Configures CORS for localhost frontend origins.
- Loads `SentenceTransformer('all-MiniLM-L6-v2')`.
- Creates a Pinecone index client.
- Creates a Groq API client.
- Sets a fixed Windows Tesseract path.

Core helper responsibilities:
- `extract_text_from_pdf`: tries text extraction with `PyPDF2`.
- `extract_text_from_scanned_pdf`: OCR fallback using `pdf2image` plus `pytesseract`.
- `extract_skills_from_text`: keyword-based fallback skill extraction.
- `get_structured_resume_json`: sends a prompt to Groq and expects strict JSON.
- `merge_skills`: combines LLM-structured skills with fallback keyword matches.
- `build_query_from_resume`: assembles a semantic search query from summary, experience, projects, and skills.
- `build_query`: creates a job-side embedding sentence such as `title at company in location with skills ...`.
- `generate_embedding`: converts text into a 384-dimensional vector.
- `save_to_pinecone`: upserts vectors and metadata.
- `search_jobs_by_query`: embeds a candidate query and retrieves matches from Pinecone.

Python AI Endpoints

`GET /`
- Simple health check returning `{"message": "API is running"}`.

`POST /generate-query`
- Input model: `JobRequest`
- Use case: called by Spring Boot after a job is saved in MySQL.
- Process: build query string, create embedding, validate dimension equals 384, upsert vector into Pinecone, return metadata and status.

`GET /search`
- Query params: `query`, optional `top_k`, optional `max_score`
- Process: embed the query and search Pinecone namespace `job-queries`
- Output: ranked matches with score and metadata

`POST /analyze-resume`
- Input: uploaded PDF
- Process:
  1. Verify file exists and ends with `.pdf`
  2. Read the file bytes
  3. Extract text using `PyPDF2`
  4. Fall back to OCR if extracted text is too short
  5. Persist extracted text to `resume/extracted_texts`
  6. Ask Groq to return a structured JSON representation
  7. Merge extracted skills
  8. Build `resume_query`
  9. Persist the structured JSON to disk
  10. Return a large JSON response with preview text and generated query

`POST /search-jobs-from-resume`
- Similar to `/analyze-resume` but proceeds further and immediately queries Pinecone for matches.
- It returns the resume query, top_k, max_score, extracted skills, structured JSON, and matches.

Behavior nuance:
`search_jobs_by_query` keeps matches only when `score < max_score`, while the frontend later keeps

Python AI Endpoints

matches only when `score > 0.35`. This means the effective acceptance window is `0.35 < score < 2.5`. The in-code UI message still says `No jobs found with score greater than 2.5`, which does not match the filter logic.

End-To-End Workflows

User registration workflow:
1. The signup modal posts registration data to `/register`.
2. The backend normalizes the email and saves the new `User`.
3. The frontend clears the modal fields and asks the user to sign in separately.

User login workflow:
1. The signin modal posts email and password to `/login`.
2. The backend compares raw password strings.
3. The backend writes authentication into the HTTP session.
4. The frontend reloads, and on boot `AuthContext` calls `/me`.
5. The frontend stores the returned user in local storage.
6. `Header` checks `/get-profile` and surfaces profile status.

Profile creation workflow:
1. Logged-in user navigates to `/add-profile`.
2. The profile form sends `phoneNumber`, `gender`, and `state` to `/add-profile`.
3. The backend resolves the current user from the session and creates a one-to-one `Profile`.
4. Subsequent header checks change from `Add Profile` to `Update Profile`.

Resume analysis workflow:
1. User uploads a PDF on `/add-profile` or `/get-jobs`.
2. Spring Boot can proxy the upload to `/analyze-resume`, or the frontend can call FastAPI directly.
3. FastAPI extracts text, falls back to OCR if needed, builds structured JSON with Groq, merges skills, and returns a generated query.

Job ingestion workflow:
1. An admin or external client calls `/create-job` on Spring Boot.
2. Java saves the job.
3. Java calls FastAPI `/generate-query`.
4. FastAPI creates an embedding and upserts metadata into Pinecone.
5. Java returns a combined `JobResponseDTO`.

Candidate matching workflow:

End-To-End Workflows

1. `SearchJobs.jsx` uploads the candidate resume to FastAPI.
2. FastAPI returns `resume_query`.
3. The frontend calls FastAPI `/search`.
4. Pinecone similarity results come back with metadata.
5. The frontend filters and renders recommended jobs.

CI/CD And Operational Files

`Jenkinsfile` defines a straightforward Windows-oriented pipeline:
- Checkout source from SCM.
- Run `where py` and `py --version`.
- Build backend with `mvnw.cmd clean install -DskipTests`.
- Build frontend with `npm install` and `npm run build`.
- Compile-check the Python service with `py -m py_compile main.py`.
- Run backend tests with `mvnw.cmd test`.
- Archive backend JARs and frontend build assets.

Operationally, this means:
- The backend and frontend are treated as first-class build artifacts.
- The Python service is syntax-checked but not executed or integration-tested.
- There is no test stage for the frontend.
- There is no deployment stage in the pipeline itself.

Other notable repository files:
- `README.md` is aspirational and more polished than the current implementation in some places.
- `copilot-instructions.md` appears to be contributor or AI-assistant guidance.
- `h.go` is a standalone Go basics example unrelated to the product.
- `resume/temp.py` is a tiny Python sandbox script unrelated to the main system.

Implementation Gaps, Risks, And Interview Talking P

Security gaps:
- Passwords are stored and compared in plaintext.
- CSRF is disabled globally.
- Session auth is used, but parts of the frontend also try to write a token that does not exist.
- Some security matchers are overly permissive compared with controller intent.

Integration gaps:
- Frontend code mixes backend ports `8080` and `8081`.
- Some flows call FastAPI through Spring Boot while others call FastAPI directly.
- Job persistence and vector persistence are not transactional together.

Data-model gaps:
- `Job.skills` is not mapped with standard JPA collection annotations.
- `ProfileRepository` uses `Integer` in the repository generic but `Profile.id` is `Long`.
- The profile form includes an `id` input that is not used by `ProfileRequestDTO`.

UX and maintainability gaps:
- Several dashboard components are placeholders returning `null`.
- `Header` contains debugging alerts and a `Check /me` button that looks like a development aid.
- `Signin.jsx` parses login response as text and then tries to use `data.token`.
- Success and filter messages in job search do not fully align with the actual threshold logic.

Strong interview talking points:
- Explain why the architecture is split between transactional CRUD in Spring Boot and semantic matching in FastAPI.
- Discuss why session-based auth was used and how you would harden it for production.
- Show that you understand the OCR fallback path for scanned resumes.
- Point out that Pinecone stores vector embeddings while MySQL stores canonical business data.
- Be honest about current limitations and describe the next refactors: password hashing, unified API client, proper entity mapping, service-to-service retry handling, and better test coverage.

Interview Prep Cheat Sheet

If you need a concise explanation during an interview, use this structure:

1. Product goal:
This project helps candidates find relevant internships by combining normal CRUD data management with AI-based resume understanding and vector similarity search.

2. Backend role:
The Spring Boot service owns users, profiles, jobs, sessions, and the main REST contract for the frontend.

3. AI service role:
The FastAPI service extracts text from resumes, uses an LLM to structure resume data, builds embeddings, and stores or searches those embeddings in Pinecone.

4. Frontend role:
The React app gives users the landing page, auth flows, profile submission flow, and resume-to-job recommendation flow.

5. Matching flow:
Jobs are embedded and indexed into Pinecone. Candidate resumes are converted into semantic queries. Pinecone returns the closest job vectors, and the frontend shows the best matches.

6. Honest limitations:
The code works as a proof of concept but still needs production-grade security, stronger consistency guarantees, and cleaner integration boundaries.